

Summarize the following paragraph in one sentence.

C is a programming language developed at AT & T's Bell
Laboratories of USA in 1972. It was designed and w

✓

Developed by Dennis Ritchie at Bell Labs in 1972, the C programming language organically grew in popularity to replace established languages of its time without any official promotion.

How is f

You stopped this response

Question: How is football played?

Answer: By passing, shooting and scoring a goal.

Now answer: How is cricket player?

Answer: By batting, bowling and scoring runs.

But alas, I still have no idea where I would even want to travel next. Shall I try the age-old trick of spinning a globe, closing my eyes and traveling to whatever random place my finger lands? This brings me to our next writing prompt, \



Let's give that globe a hearty spin, shall we? *Whirrrrr... stop!* My finger has landed squarely on the vast, mysterious continent of Artificial Intelligence.

If **Artificial Intelligence (AI)** is the entire globe—the grand, overarching dream of building machines capable of mimicking human thought and behavior—then **Machine Learning (ML)** is a specific, sprawling country nestled right in the heart of it.

Think of it like this: AI is the ultimate destination, the desire to have a brilliant travel companion who can navigate foreign streets, haggle in local markets, and maybe even write a postcard home. Early on, programmers tried to build this companion by handing it a massive, rigid guidebook containing every single rule imaginable (e.g., *if the vendor says this, then you say that*). But alas, the world is far too complex to be contained in one guidebook!

Enter Machine Learning. Instead of handing our companion a rigid itinerary, we drop them into the bustling streets of a thousand cities with a massive pile of data—conversations, maps, photos—and simply say, "Here, experience all of this and figure out the patterns yourself!"

With Machine Learning, the system learns to navigate not because we explicitly programmed every single left turn, but because it practiced analyzing millions of previous journeys until it understood *how* to find its way.

In short, Artificial Intelligence is the grand journey to create a truly smart machine, and Machine Learning is simply the most successful vehicle we've found to get us there—one that learns from the scenery as it drives!

Can you explain docker and kubernetes using the same style?

Let's grab our passports and head to the bustling, industrialized ports of the tech world!

Docker: The Perfect, Self-Contained Suitcase

If we are traveling across the digital ocean, **Docker** is the ultimate piece of luggage. In the old days, moving an application into a new hotel (a new server) was a nightmare—you'd arrive and find the electrical outlets were wrong, they didn't have your preferred soap, and the bed was too small. Your daily routine (your code) would completely break down.

Docker solves this by letting developers pack an application, along with its exact preferred plumbing, wiring, and pajamas (all of its dependencies and libraries), into one standardized container. No matter where you ship this container—a local laptop, a massive enterprise server, or a cloud on the other side of the globe—it opens up and runs exactly the same way. You never have to sigh and say, "But alas, it worked perfectly on my machine!"

Kubernetes: The Master Cruise Director

Now, what happens when your humble travel agency suddenly goes viral, and you have thousands of these perfectly packed Docker containers stacked everywhere? Chaos!

Enter **Kubernetes**. If Docker is the container, Kubernetes is the brilliant, slightly obsessive Cruise Director managing the entire fleet.

You certainly don't want to manually check if every single container is safe and in the right cabin. Instead, you simply hand Kubernetes the master itinerary and say, "I need exactly 50 of these web-server containers running at all times." Kubernetes takes the wheel. If a sudden surge of tourists (web traffic) hits, it automatically clones the containers to handle the massive crowd. If a ship (a server) happens to sink in the night, Kubernetes instantly rescues the containers and seamlessly transfers them to a healthy ship before the passengers even notice a bump.

In short: Docker packs the indestructible suitcase so your app can travel anywhere, and Kubernetes manages the fleet, ensuring thousands of those suitcases survive the turbulent seas of the internet!

Write a one-shot prompt to generate a polite response for a customer complaint, using the example given below

"Yes, I understand how frustrating this can be. Now let's take a deep breath and get to the bottom of this."

Here is a one-shot prompt you can use. It provides the AI with clear instructions and uses your provided text as the example output (the "one shot") to set the expected tone and style before giving it a new task.

Plaintext



You are a polite and empathetic customer service representative. Your task is

Example:

Customer Complaint: "I have been trying to log in for an hour and your system

Response: "Yes, I understand how frustrating this can be. Now let's take a c

Now generate a response for the following:

Customer Complaint: "My delivery is three days late and I was supposed to ha

Response:

Rewrite the above written prompt using chain-of-thought prompting technique

Here is how you can rewrite that prompt using the **Chain-of-Thought (CoT)** technique.

By adding a "Thought Process" section to the example, you teach the AI to pause and break down the emotional state of the customer and the goal of the response *before* it generates the final text. This usually results in much more accurate and empathetic replies.

Plaintext



You are a polite and empathetic customer service representative. Your task is

Example:

Customer Complaint: "I have been trying to log in for an hour and your system is down."

Thought Process: The customer is highly stressed and frustrated by a technical issue.

Response: "Yes, I understand how frustrating this can be. Now let's take a closer look at the problem."

Now generate a thought process and a response for the following:

Customer Complaint: "My delivery is three days late and I was supposed to have it yesterday."

Thought Process:

"Few-shot prompting is used when the task is very simple and well-known."

Evaluate the above statement as true or false with justifications

False.

Few-shot prompting is actually used when a task is **complex, novel, or requires a highly specific format**, not when it is simple and well-known.

Here is why the statement is incorrect:

Zero-shot is for simple tasks: If a task is very simple and well-known (e.g., basic translation, standard summarization, or factual Q&A), large language models can handle it using **zero-shot prompting**. They already understand the underlying patterns from their training data and do not need examples to succeed.

Few-shot teaches specific patterns: Few-shot prompting is deployed when you need to teach the model a unique style, tone, or formatting rule that it would not use by default. By providing a few examples, you calibrate the model's behavior to your exact specifications.

Few-shot handles ambiguity: For complex tasks where instructions alone might be misinterpreted (such as classifying nuanced sentiment or mapping custom

data inputs to a rigid data structure), showing examples clarifies edge cases and guides the model's reasoning.

In short, you use few-shot prompting to map out unfamiliar territory or strict constraints, not to walk a well-known path.

Improve the prompt below by choosing the correct technique (Zero-Shot, One-Shot, or Few-Shot).

Prompt:

“Make this sound professional: ‘I am in a meeting right now.’”

The best technique to use here is **Few-Shot Prompting**.

Because "professional" is a highly subjective term—it can range from a quick, polite ping to a highly formal email—providing a few examples is the most effective way to calibrate the AI to the exact tone, length, and style you want.

Here is the improved prompt:

Plaintext



You are a workplace communication assistant. Your task is to rewrite casual

Example 1:

Casual: "I'm running late."

Professional: "I apologize, but I am running a few minutes behind schedule."

Example 2:

Casual: "I can't look at this right now."

Professional: "I am currently focused on another task, but I would be happy

Example 3:

Casual: "I am in a meeting right now."

Professional:

Why this is an improvement:

Persona Assignment: Starting with "You are a workplace communication assistant" gives the AI immediate context for its role.

Defined Constraints: Specifying that the output should be "polite, concise, and professional" prevents the model from generating a response that is excessively long or overly formal.

Pattern Recognition: By providing two contrasting examples (Few-Shot), you establish a clear pattern of transforming blunt, casual phrases into courteous, solution-oriented professional responses.

Compare the prompts below and identify which one will give better output.

Prompt A:

"Write a product description."

Prompt B:

"Write a 3-sentence product description for a budgeting app targeting young professionals."

^

Prompt B will generate a significantly better output.

Here is a breakdown of why Prompt B is the superior prompt:

Specific Subject: It tells the AI exactly what the product is ("a budgeting app"), preventing the model from hallucinating a random product out of thin air.

Defined Audience: By specifying the target demographic ("young professionals"), the AI knows how to adjust the tone and vocabulary. It will automatically use modern, career-focused language (like "financial goals" or "salary") rather than speaking to teenagers or retirees.

Clear Constraints: Providing a strict length limit ("3-sentence") forces the AI to be concise and deliver a punchy, well-structured description.

Prompt A is far too broad. Because it lacks all context, the AI is forced to guess the product, the audience, and the format. This lack of direction almost guarantees a